

(3) 折扣累积增益 (Discounted Cumulative Gain, 简称DCG)

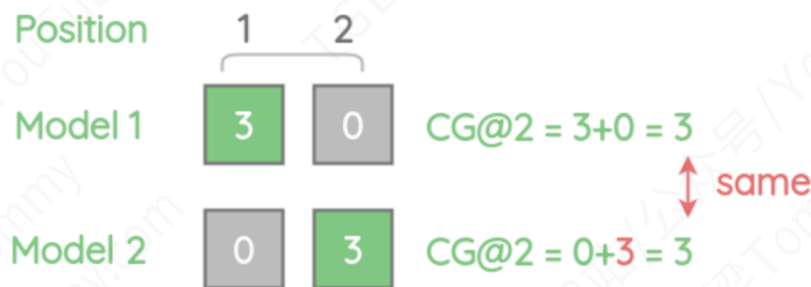
[论文](#)

[参考来源-1](#)

[参考来源-2](#)

CG是一个用于信息检索领域的评估指标，用来衡量搜索结果的相关性。

DCG是CG的一个改进版本，它通过考虑搜索结果的位置来解决CG不考虑顺序的问题。



DCG对每个位置的相关性得分应用了一个惩罚函数，这个惩罚函数基于对数函数，目的是给予靠前位置的高相关性得分更大的权重，从而反映出相关结果在搜索结果列表中的位置对用户体验的影响。

DCG的数学公式，它通过应用对数惩罚函数来降低每个位置的相关性得分：

$$DCG@k = \sum_{i=1}^k \frac{rel_i}{\log_2(i+1)}$$

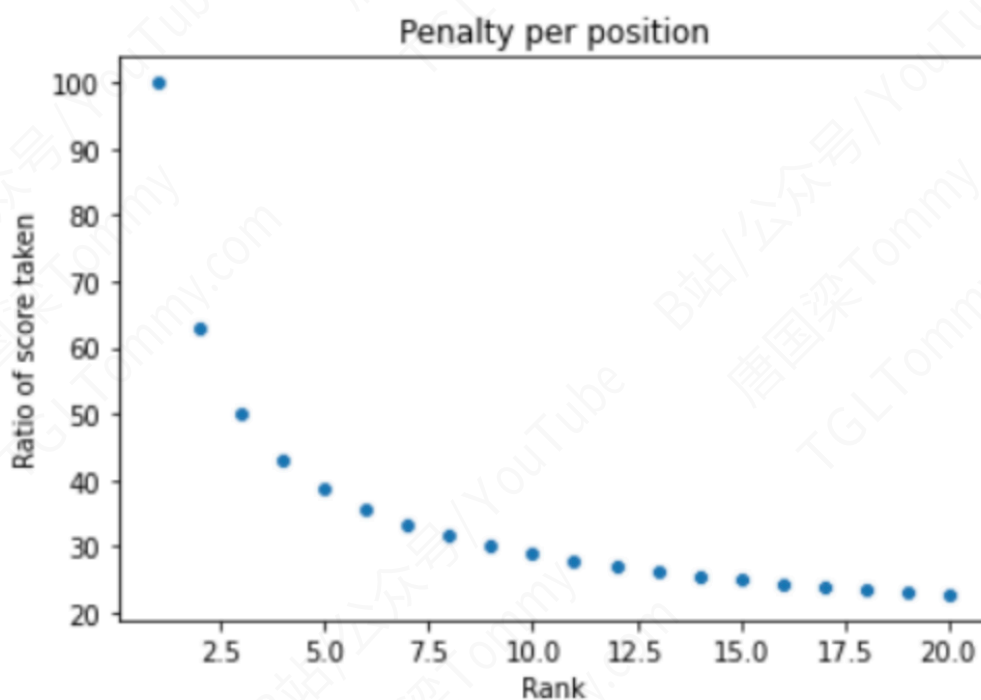
这里， rel_i 是第*i*个结果的相关性得分，而除以 $\log_2(i+1)$ 是位置*i*的惩罚项。随着位置的增加，惩罚增大，这意味着在更高的位置上获得的得分会由于更高的分母而被更多地降低。

<i>i</i>	$\log_2(i+1)$
1	$\log_2(1+1) = \log_2(2) = 1$
2	$\log_2(2+1) = \log_2(3) = 1.5849625007211563$
3	$\log_2(3+1) = \log_2(4) = 2$
4	$\log_2(4+1) = \log_2(5) = 2.321928094887362$
5	$\log_2(5+1) = \log_2(6) = 2.584962500721156$

DCG@2的计算会考虑两个结果的相关性得分，同时对第2个位置上的得分施加对数惩罚。惩罚函数的具体值如下所示：

- 对于位置1：无惩罚，因为 $\log_2(1 + 1) = \log_2(2) = 1$
- 对于位置2：惩罚为 $\log_2(2 + 1) = \log_2(3) \approx 1.58$
- ...依此类推

所以，使用这个惩罚函数，我们可以为每个位置的相关性得分计算一个折扣，反映其对用户体验的实际影响。这使得DCG成为一个更精细和实用的搜索结果相关性评估指标。



上图用以解释对数惩罚如何随着位置的增加而导致分数衰减。图表的X轴表示排名位置，Y轴表示保留的相关性得分的百分比。对于位置1，没有应用惩罚，因此得分保持不变（即100%）。但是，从位置1到位置2，保留的分数百分比下降到了63%，到位置3进一步下降到了50%，以此类推。这个衰减过程是指数的，显示了对数惩罚函数是如何工作的，更高的排名（即排名数值较大）会得到更大的惩罚。

>>> 举例讲解



$Position(i)$	$Relevance(rel_i)$	$\log_2(i + 1)$	$\frac{rel_i}{\log_2(i+1)}$
1	3	$\log_2(1 + 1) = \log_2(2) = 1$	$3 / 1 = 3$
2	2	$\log_2(2 + 1) = \log_2(3) = 1.5849625007211563$	$2 / 1.5849 = 1.2618$
3	3	$\log_2(3 + 1) = \log_2(4) = 2$	$3 / 2 = 1.5$
4	0	$\log_2(4 + 1) = \log_2(5) = 2.321928094887362$	$0 / 2.3219 = 0$
5	1	$\log_2(5 + 1) = \log_2(6) = 2.584962500721156$	$1 / 2.5849 = 0.3868$

在这个例子中，我们有五个位置的相关性得分分别是 3、2、3、0 和 1。表格中列出了每个位置的相关性得分 rel_i ，以及对应的惩罚项 $\log_2(i + 1)$ ，和计算出的 $\frac{rel_i}{\log_2(i+1)}$ 值。通过对这些值求和，我们可以得到总的 DCG 分数，该分数反映了相关性得分和结果位置的综合效果。

总的 DCG 值计算如下：

$$DCG = \frac{3}{\log_2(2)} + \frac{2}{\log_2(3)} + \frac{3}{\log_2(4)} + \frac{0}{\log_2(5)} + \frac{1}{\log_2(6)}$$

$$DCG = 3 + 1.2618 + 1.5 + 0 + 0.3868$$

$$DCG = 3 + 1.2618 + 1.5 + 0.3868$$

$$DCG \approx 6.1486$$

通过这种方式，DCG 能够有效地衡量排名质量，确保高相关性的结果排在前面可以获得更高的评分。

下面的表格列出了计算得到的 DCG 值，对于 $k=1$ 到 5，使用先前图中提供的数值。

例如， $DCG@2$ 是第一个和第二个项目的加权相关性得分之和，即 $3 + 1.2618$ ，等于 4.2618。

对于 $DCG@3$ ，则是 $DCG@2$ 的值加上第三个位置的加权分数，即 $4.2618 + 1.5 = 5.7618$ ，依此类推。

k	DCG@k
DCG@1	3
DCG@2	$3 + 1.2618 = 4.2618$
DCG@3	$3 + 1.2618 + 1.5 = 5.7618$
DCG@4	$3 + 1.2618 + 1.5 + 0 = 5.7618$
DCG@5	$3 + 1.2618 + 1.5 + 0 + 0.3868 = 6.1486$

DCG的另一种计算方式，即替代的公式：

$$DCG@k = \sum_{i=1}^k \frac{2^{rel_i} - 1}{\log_2(i+1)}$$

这个公式与之前的DCG公式的区别在于，它通过 $2^{rel_i} - 1$ 替换了 rel_i ，这样可以给排名低的相关项赋予更大的惩罚，因为指数函数的增长速度快于线性函数。



上图展示了DCG在比较不同查询结果时可能遇到的问题：

例如，如果查询Q1有3个结果，而查询Q2有5个结果，即使两个查询的前三个相关性得分是相同的，查询Q2的DCG@5可能会比查询Q1的DCG@3大。

这就引出了DCG的一个限制：不同数量结果的查询其DCG值不可直接比较，因为更多的结果可能会导致更高的DCG值，即使其平均质量不一定更高。为了解决这个问题，通常会使用规范化的DCG（Normalized DCG或NDCG），这个指标会将DCG的值规范化到一个固定的范围内（通常是0到1），从而可以更公平地比较不同查询的结果质量。

(4) 归一化折扣累积增益 (NDCG@k)

NDCG是DCG的一种归一化形式，它使得跨不同查询的DCG值能够进行比较。NDCG通过将DCG除以理想情况下的DCG（即IDCG）来实现归一化。理想DCG是指将相关项按照其相关性得分的降序排列时的DCG值。

归一化操作的公式如下：

$$NDCG@k = \frac{DCG@k}{IDCG@k}$$

在给定的示例中，先前已经计算了各个k值下的DCG，现在，要计算NDCG，我们需要确定每个k值下的理想DCG：

1. 首先对所有相关性得分进行降序排列。
2. 然后计算这些排好序的得分的DCG值，这将成为IDCG。
3. 最后，将原DCG@k除以对应的IDCG@k得到NDCG@k。



k	DCG@k
DCG@1	3
DCG@2	$3 + 1.2618 = 4.2618$
DCG@3	$3 + 1.2618 + 1.5 = 5.7618$
DCG@4	$3 + 1.2618 + 1.5 + 0 = 5.7618$
DCG@5	$3 + 1.2618 + 1.5 + 0 + 0.3868 = 6.1486$

例如，如果我们的理想顺序下的相关性得分是3、3、2，那么对于DCG@3，IDCG@3将是：

$$IDCG@3 = \frac{3}{\log_2(2)} + \frac{3}{\log_2(3)} + \frac{2}{\log_2(4)}$$

$$IDCG@3 = 3 + 1.8928 + 1$$

$$IDCG@3 = 5.8928$$

因此，对于DCG@3 = 5.7618，我们可以得到：

$$NDCG@3 = \frac{5.7618}{5.8928}$$

这个值将介于0和1之间，允许我们比较不同查询的性能。NDCG的优点在于它可以标准化不同长度的查询结果列表，使得它们之间可以进行公平的比较。

如何计算理想DCG (IDCG) 以及如何进一步计算NDCG

如何计算理想情况下的DCG，即IDCG，其中项目按照相关性分数的降序排列。计算方法与DCG相同，只是相关性得分按照最优顺序排列。



如何使用IDCG来计算NDCG：

$Position(i)$	$Relevance(rel_i)$	$\log_2(i+1)$	$\frac{rel_i}{\log_2(i+1)}$	IDCG@k
1	3	$\log_2(2) = 1$	$3 / 1 = 3$	3
2	3	$\log_2(3) = 1.5849$	$3 / 1.5849 = 1.8927$	$3 + 1.8927 = 4.8927$
3	2	$\log_2(4) = 2$	$2 / 2 = 1$	$3 + 1.8927 + 1 = 5.8927$
4	1	$\log_2(5) = 2.3219$	$1 / 2.3219 = 0.4306$	$3 + 1.8927 + 1 + 0.4306 = 6.3233$
5	0	$\log_2(6) = 2.5849$	$0 / 2.5849 = 0$	$3 + 1.8927 + 1 + 0.4306 + 0 = 6.3233$

对于每个k值，都显示了相应的DCG@k、IDCG@k和NDCG@k的计算和结果。

$$NDCG@k = \frac{DCG@k}{IDCG@k}$$

NDCG通过将DCG除以IDCG得到，其值介于0到1之间。理想的排名将得到NDCG分数为1。这样就可以跨查询比较NDCG得分，因为它是一个归一化的得分。

k	DCG@k	IDCG@k	NDCG@k
1	3	3	$3 / 3 = 1$
2	4.2618	4.8927	$4.2618 / 4.8927 = 0.8710$
3	5.7618	5.8927	$5.7618 / 5.8927 = 0.9777$
4	5.7618	6.3233	$5.7618 / 6.3233 = 0.9112$
5	6.1486	6.3233	$6.1486 / 6.3233 = 0.9723$

例如，对于DCG@3，我们有：

$$DCG@3 = 5.7618$$

$$IDCG@3 = 5.8927$$

$$NDCG@3 = \frac{DCG@3}{IDCG@3} = \frac{5.7618}{5.8927} = 0.9777$$

这意味着对于前三个项目，查询的排名质量接近于理想情况。NDCG分数越接近于1，表明排名质量越好。这种方法允许我们将具有不同数量结果的查询进行公平比较。

```
import numpy as np

def dcg_at_k(scores, k=5):
    """
    计算DCG@k。
    参数：
    - scores: 一个包含相关性得分的列表或者numpy数组。
    - k: 考虑的顶部元素的数量。
    返回：
    - DCG@k的值。
    """
    scores = np.asarray(scores)[:k]
    return np.sum((2**scores - 1) / np.log2(np.arange(2, scores.size + 2)))

def idcg_at_k(scores, k=5):
    """
    计算IDCG@k。
    参数：
    - scores: 一个包含相关性得分的列表或者numpy数组。
    - k: 考虑的顶部元素的数量。
    """
```

```
返回：
- IDCG@k的值。
"""

scores_sorted = np.sort(scores)[::-1]
return dcg_at_k(scores_sorted, k)

def ndcg_at_k(scores, k=5):
    """
    计算NDCG@k。
    参数：
    - scores：一个包含相关性得分的列表或者numpy数组。
    - k：考虑的顶部元素的数量。
    返回：
    - NDCG@k的值。
    """

    dcg = dcg_at_k(scores, k)
    idcg = idcg_at_k(scores, k)
    return dcg / idcg if idcg > 0 else 0

# 示例使用
scores = [3, 2, 3, 0, 1]
k = 3
ndcg_score = ndcg_at_k(scores, k)
print(f"NDCG@{k}: {ndcg_score}")

'''输出值

NDCG@3: 0.9594535145926796
'''
```

1.1.2 生成模块评估

- 生成模块的评估关注于检索到的文档如何增强输入，以形成对查询的响应。与端到端评估中的最终答案/响应生成不同，这里更侧重于评估过程中的上下文相关性。
- 评估指标主要包括BLEU、ROUGE等，这些都是衡量文本生成质量的常用指标。特别是，它们能够评估生成文本的流畅度、一致性以及与原始查询问题的相关性。

(1) BLEU

BLEU (Bilingual Evaluation Understudy) 得分的计算方法，这是一种评估机器翻译质量的指标。它通过比较机器翻译的输出和一组参考翻译来计算得分。

BLEU得分计算步骤

步骤 1: 理解BLEU得分的定义